## Agenda:

1. Pair sum
2. Count of Pair Sum
3. Check if there exits a subarray with sum = 0
4. Longest Subarray with sum = 0
5. Check if there exits a subarray with sum = K

**Q1:** Given an integer array **A[ ]** and an integer **K**.

Check if there exist a pair (i, j) such that **A[i] + A[j] = K** and **i != j**.

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| Array | 8 | 9 | 1 | -2 | 4 | 5 | 11 | -6 | 4 |

K = 6: arr[2] + arr[5] : such a pair exists

K = 22: does not exist

K = 8: arr[4] + arr[8]: such a pair exists

2 Pointer Solution → Sort → $TC = O(N \log(N))$
Sol of 2 pointers → $TC = O(N)$
} $TC = O(N \log(N))$

Bruteforce → $\forall i, j$ check if $A[i] + A[j] == K$ && $i != j$.

$TC = O(N^2)$ $SC = O(1)$

Idea → $A[i] + A[j] = K \Rightarrow \boxed{A[j] = K - A[i]}$

Sol → $\forall i$, calculate $\boxed{K - A[i]}$ & check if $(K - A[i])$ is present in A[].

Search in unsorted array → Hash Set

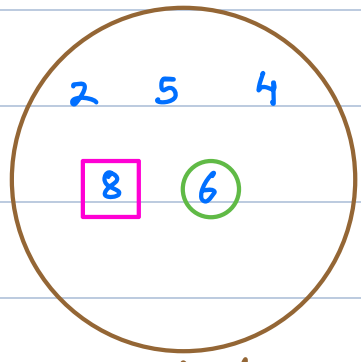
Sorted → Binary Search → $TC = O(N \log(N))$

**Sol 1** → 1) Insert all elements in hashset

2) $\forall i$, check if $K - A[i]$ is present in hashset.

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| A = [ | 2 | 5 | 4 | 8 | 2 | 6 ] |
| | i | | | j | | |

$K = 14$




Hash Set

**check** →

$14 - 2 = 10$ ✗

$14 - 5 = 9$ ✗

$14 - 4 = 10$ ✗

$14 - 8 = 6$ ✓ Ans = true

$K = 16$

$16 - 2 = 14$ ✗

$16 - 5 = 11$ ✗

$16 - 4 = 12$ ✗

$16 - 8 = 8$ ✗ $(i == j)$

**Sol 2** $i \,!= j$ → $i < j$ or $\boxed{i > j}$

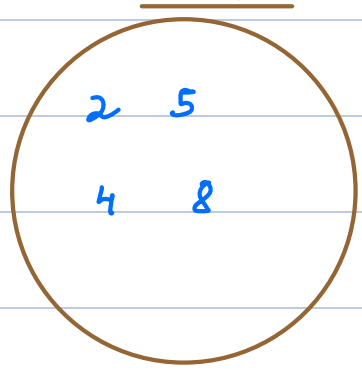
$\forall i$, check if $K - A[i]$ is present in HashSet &

after check insert $A[i]$ in HashSet.

| | i | i | i | i | i | j |
|---|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 | 5 |
| A = [ | 2 | 5 | 4 | 8 | 2 | 6 ] |

$K = 16$

HashSet




**check** →

$16 - 2 = 14$

$16 - 5 = 11$

$16 - 4 = 12$

$16 - 8 = 8$

$16 - 2 = 14$

$16 - 6 = 10$

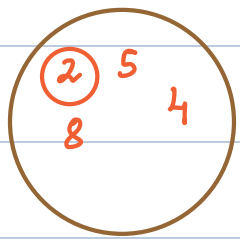
Ans = false

```
for i → 0 to (N-1) {
    if (hs.contains (K - A[i]))  return true
    hs.add (A[i])
}
return false
```

TC = O(N)  SC = O(N)

| | | | | | |
|---|---|---|---|---|---|
| ~~i~~ | ~~i~~ | ~~i~~ | ~~i~~ | i | |
| 0 | 1 | 2 | 3 | 4 | 5 |

A = [ 2  5  4  8  2  6 ]  K = 4




check → 4 - 2 = 2

4 - 5 = -1

4 - 4 = 0

4 - 8 = -4

4 - 2 = 2 ✓

Ans = true

**Q2:** Given an integer array **A[ ]** and an integer **K**.

Count the number of pairs such that **A[i] + A[j] = K** and **i != j**.

Example: A = [ 2 5 2 8 5 10 ] K = 10

(0 1 2 3 4 5)

$(i, j) \rightarrow (0,3), (2,3), (1,4)$ Ans = 3

Sol → $\forall i$, if K-A[i] is present → ans += freq of (K-A[i]).
After check increase frequency of A[i]. $(j < i)$

A = [ 2 5 2 8 5 10 ] K = 10
(0 1 2 3 4 5)
(j j j j j j i)

HashMap
2 → 2
5 → 2
8 → 1
10 → 1

check → 10 - 2 = 8
10 - 5 = 5
10 - 2 = 8
10 - 8 = 2
10 - 5 = 5
10 - 10 = 0

ans = 0 + 2 + 1
= 3

```
ans = 0

for i -> 0 to (N-1) {
    if (hm.containsKey (K - A[i]))
        ans += hm.get (K - A[i])
    f = hm.getOrDefault (A[i], 0)
    hm.put (A[i], f+1)
}

return ans
```

TC = O(N)  SC = O(N)

**Q3:** Given an integer array **A[ ]**. Check if there exists a subarray with **sum = 0**.

Example: A = [ 2 2 1 -3 1 3 1 -2 -3 2 ]

Ans = true

subarray sum → Prefix sum

subarray sum i — j = P[j] - P[i-1]

↳ P[j] , (i == 0)

P[j] - P[i-1] = 0 ⇒ $\boxed{P[j] = P[i-1]}$

P[i] = P[i-1] + A[i]

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| A = [ | 2 | 2 | 1 | -3 | 1 | 3 | 1 | -2 | -3 | 2 ] |
| P → | 2 | 4 | 5 | 2 | 3 | 6 | 7 | 5 | 2 | 4 |

Sol → ∀ i, check if P[i] is present in P[] array on left.

```
for i → 0 to (N-1) {
    if (P[i] == 0) return true
    if (hs.contains(P[i])) return true
    hs.add(P[i])
}
return false
```

| | 0 | 1 | 2 |
|---|---|---|---|
| A = [ | 2 | 5 | -7 ] |
| P → | 2 | 7 | 0 |

```
p = 0
for i → 0 to (N-1) {
    p = p + A[i]
    if (p == 0) return true
    if (hs.contains(p)) return true
    hs.add(p)
}
return false
```

SC = O(N)  TC = O(N)

**Q4:** Given an integer array **A[ ]**. Find the length of longest subarrays with **sum = 0**.

If none exist → return **0**.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| Example: A = [ | 2 | 2 | 1 | -3 | 1 | 3 | 1 | -2 | -3 | 2 ] |

Ans = 8

$3-0=3$ $\quad 7-2=5$ $\quad 8-0=8$ ✓

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| A = [ | 2 | 2 | 1 | -3 | 1 | 3 | 1 | -2 | -3 | 2 ] |
| P → | 2 | 4 | 5 | 2 | 3 | 6 | 7 | 5 | 2 | 4 |
| | | | | i | | | | i | i | i |

$9 \rightarrow 9-1=8$ ✓

$8-0=8$ (Ans)

$2 \rightarrow \{0, 3, 8\}$ all index ✗

$2 \rightarrow \{0\}$ first index $\quad L \longrightarrow R$ ✓

$2 \rightarrow \{8\}$ last index $\quad L \longleftarrow R$

Sol → $\forall i$, check if $P[i]$ is present on left,

if yes → $i$ − (first index of $A[i]$) can be potential ans.

$A[i] \rightarrow$ first index ⇒ HashMap

if $(P[i] == 0) \Rightarrow (i+1)$ is a potential ans.

```
ans = 0        p = 0
for i -> 0 to (N-1) {
    p = p + A[i]
    if (p == 0)    ans = max (ans, i+1)
    if (hm.containsKey (p))    ans = max (ans, i - hm.get(p))
    else    hm.put (p, i)
}
return ans
```

TC = O(N)  SC = O(N)

**Q3:** Given an integer array **A[ ]**. Check if there exists a subarray with **sum = K**.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Example: A = [ | 2 | 2 | 1 | -3 | 1 | 3 | 1 | -2 | -3 | 2 ] | K = **-4** |

Ans = true

subarray sum $i - j$ $\Rightarrow$ $P[j] - P[i] = K$ $\quad$ $\boxed{i <= j}$

$\hookrightarrow$ $P[j] = K$ , $i = 0$ ✓

$P[j] = K + P[i]$ ✓ $L \leftarrow R$

OR, $P[i] = P[j] - K$ ✓ $L \rightarrow R$ ✓

```
p = 0
for i → 0 to (N-1) {
    p = p + A[i]
    if (p == K)  return true
    if (hs.contains(p-K))  return true
    hs.add(p)
}
return false
```

TC = O(N) $\quad$ SC = O(N)

**What will be the outpout for following code snippet?**

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {

        int[] arr = {1, 1, 1, 1};
        int K = 2;

        HashMap<Integer, Integer> map = new HashMap<>();
        int count = 0;+1+2+3 = 6

        for (int x : arr) {
            if (map.containsKey(K - x)) {
                count += map.get(K - x);
            }
            map.put(x, map.getOrDefault(x, 0) + 1);
        }
        System.out.println(count);
    }
}
```

1 → 1 2 3 4